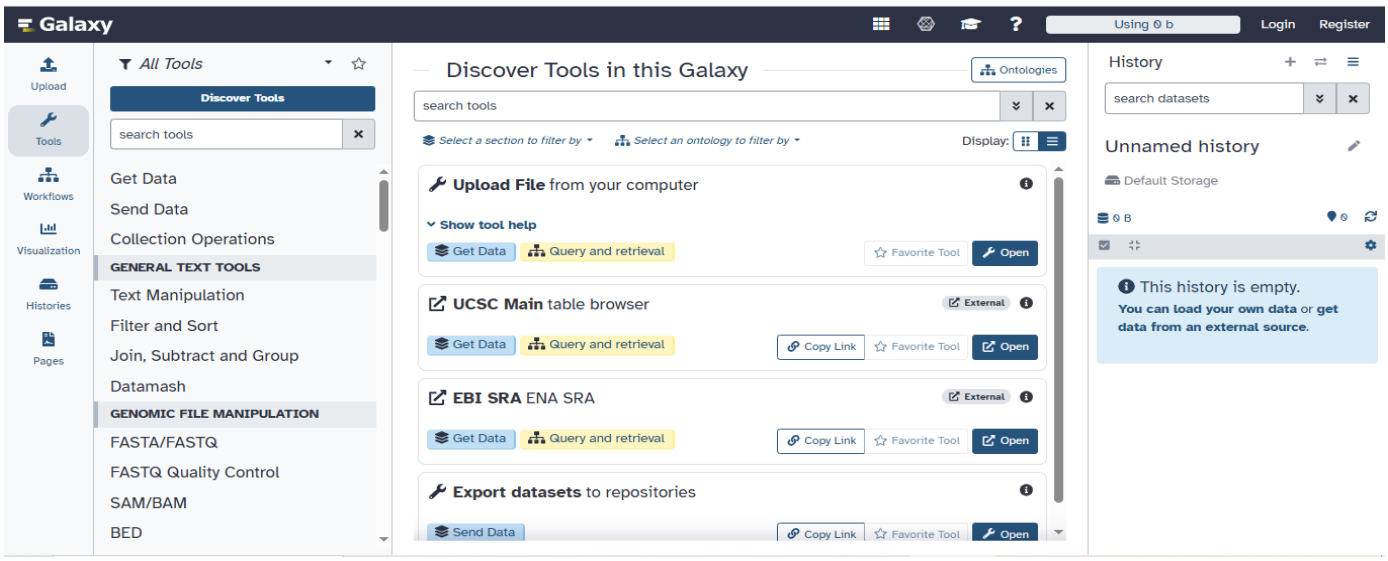
Task 1 –Overview of galaxy framework

The Galaxy Framework is a web-based platform for bioinformatics and genome data analysis. It can be accessed through a web browser, locally on a server or via public Galaxy servers making it flexible for users worldwide.
According to a 2019 presentation there are 127 publicly accessible Galaxy servers allowing researchers to use Galaxy without installing it locally.

Galaxy is beginner-friendly and can be used by anyone without programming or coding experience. Its intuitive interface displays tools on the left panel while the history panel on the right automatically records all uploaded files, outputs and results.



Galaxy allows users to create personal accounts. During registration users provide an email ID and a public name. A confirmation link is sent to their email inbox. Once activated the account saves all histories, workflows and uploaded files enabling users to continue their work at any time without losing progress.

A key feature of Galaxy is workflow creation which allows multiple analysis steps to be combined into automated pipelines. Workflows can be saved, reused and shared making Galaxy valuable for repeated experiments, teaching and collaborative research. The platform integrates thousands of community-developed tools for genomics, transcriptomics, proteomics, metagenomics and general data analysis supporting large datasets generated by modern technologies.

Using Galaxy is straightforward. Users upload data, select a tool, set the parameters, click Run and view the results in the history panel. This structured process ensures clarity, reproducibility and transparency. Galaxy also organizes data efficiently through directories and the history panel helping users track every step.

In addition to research, Galaxy is widely used in education and training as workflows can be reused and shared to teach bioinformatics. While the platform is easy to use, running Galaxy locally or on a server requires sufficient RAM, CPU and disk space. When using publicly accessible Galaxy servers local system requirements are minimal.

Summary - Galaxy is a powerful, user-friendly and flexible platform that combines accessibility with advanced functionality. Its registration system, automatic tracking, workflow creation, tool integration and reproducible analysis make it an essential resource for both beginners and experienced researchers.

Task 2 Galaxy framework installation- problem solving process

Objective

The objective of this task is to explain the thought process, steps attempted, system considerations, and challenges encountered during the Galaxy Framework installation, rather than achieving a fully functional installation. The focus is on practical problem-solving, learning outcomes, and system planning.

1. Operating System and Linux Environment Selection

Galaxy Framework is primarily designed to run on Linux-based systems. Since my primary system was Windows, it was necessary to create a Linux-compatible environment.

Approach chosen:

- Windows Subsystem for Linux (WSL) 2 was used to run a Linux environment directly on Windows without dual-booting.
- Selected Ubuntu 22.04.5 LTS as the Linux distribution

Reasons for choosing Ubuntu 22.04.5 LTS:

- Officially supported and stable Linux distribution
- Long-term support (LTS) version ensures reliability
- Widely used in bioinformatics research
- Extensive documentation and community support available
- Fully compatible with Galaxy Framework

This setup allowed a Linux-like experience while retaining the convenience of Windows.

2. Minimum System and Software Prerequisites

Hardware Requirements

- 64-bit system architecture
- Minimum RAM: 8 GB (16 GB recommended for smoother operation)
- Disk space: 20–30 GB
- Stable internet connection

Software Requirements

- Linux environment: Ubuntu 22.04.5 via WSL
- Python 3 and venv for virtual environment management
- Git for cloning Galaxy source code
- pip (Python package manager)
- Node.js and npm for client-side dependencies
- Basic Linux build tools (make, gcc, etc.)
- Web browser to access the Galaxy interface

These components were selected based on Galaxy's official documentation and community recommendations.

3. Installation Method Planning

Several installation approaches were considered:

- Manual installation from source code
- Installation using Python virtual environment
- Docker-based installation
- Cloud-based deployment
- WSL-based installation on Windows

Chosen Method:

- Manual installation on Ubuntu 22.04.5 LTS via WSL

- Justification:

  - Allowed learning of internal workings of Galaxy
  - Suitable for development and academic purposes
  - Avoided dual-booting while providing a Linux-compatible environment
  - Enabled control over dependencies and installation steps

4. Installation Steps Performed

The installation process followed these steps:

- Accessed Ubuntu 22.04.5 LTS through WSL
- Updated system packages
- Verified Git and Python installation
- Cloned Galaxy source code from GitHub
- Created and activated a Python virtual environment
- Installed Python dependencies using pip install -r requirements.txt
- Installed Node.js and npm and tested client-side dependencies
- Attempted to run Galaxy using ./run.sh

Additionally, attempts were made to manage client build complexity and resource usage:

- Skipped client build to reduce load: export GALAXY_SKIP_CLIENT_BUILD=1
- Increased Node memory allocation: export NODE_OPTIONS="--max-old-space-size=4096"

jaspreet@DESKTOP-OLQEUC6: ~

```
Setting up libalgorithm-merge-perl (0.08-3) ...
Setting up libtsan0:amd64 (11.4.0-1ubuntu1~22.04.2) ...
Setting up cpp-11 (11.4.0-1ubuntu1~22.04.2) ...
Setting up dpkg-dev (1.21.1ubuntu2.6) ...
Setting up libjs-sphinxdoc (4.3.2-1) ...
Setting up libgcc-11-dev:amd64 (11.4.0-1ubuntu1~22.04.2) ...
Setting up gcc-11 (11.4.0-1ubuntu1~22.04.2) ...
Setting up cpp (4:11.2.0-1ubuntu1) ...
Setting up libc6-dev:amd64 (2.35-0ubuntu3.11) ...
Setting up gcc (4:11.2.0-1ubuntu1) ...
Setting up libexpat1-dev:amd64 (2.4.7-1ubuntu0.6) ...
Setting up libstdc++-11-dev:amd64 (11.4.0-1ubuntu1~22.04.2) ...
Setting up zlib1g-dev:amd64 (1:1.2.11.dfsg-2ubuntu9.2) ...
Setting up g++-11 (11.4.0-1ubuntu1~22.04.2) ...
Setting up libpython3.10-dev:amd64 (3.10.12-1~22.04.12) ...
Setting up python3.10-dev (3.10.12-1~22.04.12) ...
Setting up g++ (4:11.2.0-1ubuntu1) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mode
Setting up build-essential (12.9ubuntu3) ...
Setting up libpython3-dev:amd64 (3.10.6-1~22.04.1) ...
Setting up python3-dev (3.10.6-1~22.04.1) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
jaspreet@DESKTOP-OLQEUC6:~$ git --version
git version 2.34.1
jaspreet@DESKTOP-OLQEUC6:~$ python --version
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
jaspreet@DESKTOP-OLQEUC6:~$
```

jaspreet@DESKTOP-OLQEUC6: ~/galaxy

```
      Successfully uninstalled setuptools-59.6.0
Successfully installed a2wsgi-1.10.8 adal-1.2.7 aiobotocore-2.21.1 aiofiles-24.1.0 aiohappyeyeballs-2.6.1 aiohttp-3.11.18 aioitertools-0.12.0 aiosignal-1.3.2 alembic-1.
15.2 amqp-5.3.1 annotated-types-0.7.0 anyio-4.9.0 apispec-6.8.1 appdirs-1.4.4 arcp-0.2.1 argcomplete-3.6.2 async-timeout-5.0.1 attmap-0.13.2 attrs-25.3.0 babel-2.17.0 b
agit-1.8.1 bagit-profile-1.3.1 bcrypt-4.3.0 bdbag-1.7.3 beaker-1.13.0 billiard-4.2.1 bioblend-1.5.0 bleach-6.2.0 boltons-25.0.0 boto-2.49.0 botocore-1.37.1 bx-python-0.
13.0 cachecontrol-0.14.2 celery-5.5.2 certifi-2025.4.26 cffi-1.17.1 charset-normalizer-3.4.1 circus-0.19.0 click-8.1.8 click-didyoumean-0.3.1 click-plugins-1.1.1 click-
repl-0.3.0 cloudauthz-0.6.0 cloudbridge-3.2.0 coloredlogs-15.0.1 conda-package-streaming-0.11.0 cryptography-44.0.2 ct3-3.4.0 cwl-upgrader-1.2.12 cwl-utils-0.37 cwltool
-3.1.20250110105449 defusedxml-0.7.1 deprecated-1.2.18 deprecation-2.1.0 dictobj-0.4 dnspython-2.7.0 docopt-0.6.2 docutils-0.21.2 dparse-0.6.4 edam-ontology-1.25.2 emai
l-validator-2.2.0 exceptiongroup-1.2.2 fastapi-0.115.12 filelock-3.18.0 fissix-24.4.24 frozenlist-1.6.0 fs-2.4.16 fsspec-2025.3.2 future-1.0.0 gravity-1.1.1 greenlet-3.
2.1 gunicorn-23.0.0 gxformat2-0.20.0 h11-0.16.0 h5grove-2.3.0 h5py-3.13.0 httpcore-1.0.9 httpx-0.28.1 humanfriendly-10.0 idna-3.10 importlib-metadata-8.6.1 importlib-re
sources-6.5.2 isa-rwval-0.10.11 isal-1.7.2 isodate-0.7.2 jinja2-3.1.6 jmespath-1.0.1 jsonref-1.1.0 jsonschema-4.23.0 jsonschema-specifications-2025.4.1 kombu-5.5.3 lago
m-2.7.5 limits-5.1.0 lxml-5.4.0 mako-1.3.10 markdown-3.8 markdown-it-py-3.0.0 markupsafe-3.0.2 mdurl-0.1.2 mercurial-7.0.1 mistune-3.0.2 mrcfile-1.5.4 msal-1.32.3 msgpa
ck-1.1.0 multidict-6.4.3 mypy-extensions-1.1.0 networkx-3.4.2 nodeenv-1.9.1 numpy-2.2.5 oauthlib-3.2.2 orjson-3.10.16 oyaml-1.0 packaging-25.0 paramiko-3.5.1 parsley-1.
3 paste-3.10.1 pastedeploy-3.1.0 pebble-5.1.1 pillow-11.2.1 prompt-toolkit-3.0.51 propcache-0.3.1 prov-1.5.1 psutil-7.0.0 pulsar-galaxy-lib-0.15.9 pycparser-2.22 pycryp
todome-3.22.0 pydantic-2.11.3 pydantic-core-2.33.1 pydantic-tes-0.2.0 pydot-2.0.0 pyeventsystem-0.1.0 pyfaidx-0.8.1.3 pygments-2.19.1 pyjwt-2.10.1 pykwalify-1.8.0 pylib
magic-0.5.0 pynacl-1.5.0 pyparsing-3.2.3 pypng-0.20220715.0 pysam-0.23.0 python-dateutil-2.9.0.post0 python-magic-0.4.27 python-multipart-0.0.20 python-slugify-8.0.4 py
thon3-openid-3.2.0 pytz-2025.2 pyyaml-6.0.2 pyzmq-26.4.0 rdflib-7.1.4 referencing-0.36.2 refgenconf-0.12.2 regex-2024.11.6 repoze-lru-0.7 requests-2.32.3 requests-oauth
lib-2.0.0 requests-toolbelt-1.0.0 requests-unixsocket-0.4.1 rich-13.9.4 rich-argparse-1.7.0 rocrate-0.13.0 routes-2.5.1 rpds-py-0.24.0 ruamel-yaml-0.18.10 ruamel-yaml-c
lib-0.2.12 s3fs-2025.3.2 schema-salad-8.9.20250408123006 setuptools-79.0.1 setuptools-scm-5.0.2 six-1.17.0 slowapi-0.1.9 sniffio-1.3.1 social-auth-core-4.6.0 sortedcont
ainers-2.4.0 spython-0.3.14 sqlalchemy-2.0.40 sqlitedict-2.1.0 sqlparse-0.5.3 starlette-0.46.2 starlette-context-0.4.0 supervisor-4.2.5 svgwrite-1.4.3 tenacity-9.1.2 te
xt-unidecode-1.3 tifffile-2025.3.30 tinydb-4.8.2 tomli-2.2.1 tornado-6.4.2 tqdm-4.67.1 tuspy-1.1.0 tuswsgi-0.5.5 typing-extensions-4.13.2 typing-inspection-0.4.0 tzdata
-2025.2 tzlocal-5.3.1 ubiquerg-0.8.1 urllib3-2.4.0 uvicorn-0.34.2 uvloop-0.21.0 vine-5.1.0 wcwidth-0.2.13 webencodings-0.5.1 webob-1.8.9 whoosh-2.7.4 wrapt-1.17.2 yacma
n-0.9.3 yarl-1.20.0 zipp-3.21.0 zipstream-new-1.1.8 zstandard-0.23.0
Trying to resolve path for the 'email_ban_file' option but it's empty/None
Installing node into /home/jaspreet/galaxy/.venv with nodeenv.

added 1 package in 5s/home/jaspreet/galaxy/.venv/bin/activate
npm notice
npm notice New major version of npm available! 10.9.2 -> 11.7.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.7.0
npm notice To update run: npm install -g npm@11.7.0
npm notice
yarn install v1.22.22
[1/4] Resolving packages...
warning Resolution field "chokidar@3.5.3" is incompatible with requested version "chokidar@^4.0.1"
warning Resolution field "chokidar@3.5.3" is incompatible with requested version "chokidar@^3.6.0"
[2/4] Fetching packages...
[3/4] Linking dependencies...
warning "dexie > karma-safari-launcher@1.0.0" has unmet peer dependency "karma@>=0.9".
warning " > vue-rx@6.2.0" has incorrect peer dependency "rxjs@^6.0.0".
warning " > autoprefixer@10.4.16" has unmet peer dependency "postcss@^8.1.0".
warning " > jsdom-worker@0.3.0" has unmet peer dependency "node-fetch@*".
warning " > postcss-loader@7.3.3" has unmet peer dependency "postcss@^7.0.0 || ^8.0.1".
warning Workspaces can only be enabled in private projects.
[##################################################################################################################################-------] 115528/121085_
```

jaspreet@DESKTOP-OLQEUC6: ~/galaxy

```
npm notice
yarn install v1.22.22
[1/4] Resolving packages...
warning Resolution field "chokidar@3.5.3" is incompatible with requested version "chokidar@^4.0.1"
warning Resolution field "chokidar@3.5.3" is incompatible with requested version "chokidar@^3.6.0"
[2/4] Fetching packages...
[3/4] Linking dependencies...
warning "dexie > karma-safari-launcher@1.0.0" has unmet peer dependency "karma@>=0.9".
warning " > vue-rx@6.2.0" has incorrect peer dependency "rxjs@^6.0.0".
warning " > autoprefixer@10.4.16" has unmet peer dependency "postcss@^8.1.0".
warning " > jsdom-worker@0.3.0" has unmet peer dependency "node-fetch@*".
warning " > postcss-loader@7.3.3" has unmet peer dependency "postcss@^7.0.0 || ^8.0.1".
warning Workspaces can only be enabled in private projects.
[4/4] Building fresh packages...
Done in 4218.60s.
yarn run v1.22.22
$ NODE_ENV=production gulp && yarn run webpack-production-maps && yarn run stage-build
[21:13:03] Using gulpfile ~/galaxy/client/gulpfile.js
info No lockfile found.fault'...
[1/4] Resolving packages...
warning parcel > @parcel/config-default > @parcel/optimizer-svgo > svgo > stable@0.1.8: Modern JS already guarantees Array#sort() is a stable sort, so this library is d
eprecated. See the compatibility table on MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort#browser_compatibility
[2/4] Fetching packages...
[3/4] Linking dependencies...
warning Workspaces can only be enabled in private projects.
[4/4] Building fresh packages...
success Saved lockfile.
Building annotate_image
$ yarn build-css && yarn build-js
$ cp -f 'node_modules/jquery-contextmenu/dist/jquery.contextMenu.css' 'static/'
$ parcel build src/script.js --dist-dir static
(node:5698) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
✨ Built in 3.63s

static/script.js    351.81 KB    1.10s
Successfully built, saving build state to ../config/plugins/visualizations/annotate_image/static/plugin_build_hash.txt
No build hashfile detected for fits_graph_viewer, generating now.
Installing Dependencies for fits_graph_viewer
info No lockfile found.
[1/4] Resolving packages...
warning astrovisjs > mathjax-full@3.2.2: Version 4 replaces this package with the scoped package @mathjax/src
[2/4] Fetching packages...
[#####################------------------------------------------------------------------------------------------------------------------------------] 25/191
```

5. Problems Encountered During Installation

Despite following recommended procedures, several challenges arose:

a. RAM and Resource Limitations

- WSL allocated limited memory by default.
- Galaxy is resource-intensive, especially during npm install and client build.
- Even after increasing Node memory, the system struggled, and processes often failed.

b. Client Build Issues

- The Galaxy client build required significant memory.
- Skipping client build helped partially, but the server still failed to start reliably.

c. Server Accessibility Issues

- After running ./run.sh, the Galaxy server process did not remain active.
- Accessing http://127.0.0.1:8080 resulted in "site can't be reached."

d. WSL Networking and Process Management

- Using ps aux | grep galaxy, it was observed that the Galaxy process either did not run or terminated unexpectedly.
- Likely causes: memory exhaustion and WSL process limitations.

e. Background Tasks and Dependencies

- Automatic dependency installations and background processes ran slowly.
- File service fetching and synchronization tasks caused additional delays.

6. Key Challenges Identified

- Insufficient RAM for runtime (Galaxy is memory-heavy)
- WSL resource constraints on Windows
- Large-scale framework complexity
- Heavy client and background dependency processing

7. Reasoning and Learning Outcomes

Even though the installation was not fully successful, the process provided valuable learning:

- Understanding system resource requirements for Galaxy (RAM, CPU)
- Insight into WSL limitations when running large bioinformatics frameworks
- Awareness of installation method selection based on hardware and environment
- Problem-solving skills using logs, process monitoring, and environment variables
- Hands-on experience with Linux commands, virtual environments, and dependency management

8. Preferred Approach for Future Installations

- Based on the challenges encountered, a more stable installation would include:
- Native Linux system or cloud-based VM (virtual machine)
- 8–16 GB RAM minimum for smooth operation
- Docker-based installation to simplify setup and resource management
- Choosing cloud instances for scalability and performance

Conclusion- This task demonstrates a realistic and practical approach to Galaxy Framework installation on a Windows system using WSL.

Key takeaways:

- Installation failures are often linked to hardware limitations and environment constraints
- WSL is useful for testing and learning but may struggle with resource-intensive frameworks
- A thorough problem-solving process, combined with reasoning and alternative planning, is critical in managing complex bioinformatics tools

Despite the installation not being fully successful, the exercise enhanced understanding of Linux environments, system planning, and Galaxy Framework requirements, providing a solid foundation for future installations.